

Grep

Definition

- Grep is used to search text in a given file(s). Grep works line by line. (it matches the search criteria in a line by line basis)

Usage/formula

- `grep + option = search criteria + files`

• Examples

- Find word ignoring case

- `grep -i "mo" users.txt`

- Search any line that contains the word "dracula" in the given file:

- `grep 'dracula' ~/Documents/dracula.txt`

- Search any line that contains any word that contains "dracula" regardless of the case.

- `grep -i 'dracula' ~/Documents/Books/dracula.txt`

- Display how many lines contain the match string

- `grep -c 'dracula' ~/Documents/Books/dracula.txt`

- Search any line that contains the word dracula regardless of case with line number

- `grep -in 'dracula' ~/Documents/Books/dracula.txt`

- Search all the lines that do not contain the word 'war'

- `grep -v 'war' ~/Documents/Books/war-and-peace.txt`

- Search and display only the matching string(pattern)

- `grep -o 'pride' ~/Documents/Books/war-and-peace.txt`

- Display a list of users with the /bin/bash/ login shell

- `grep -i "bin/bash" /etc/passwd`

- Display the user's information as stored in the /etc/passwd

- `grep -i $USER /etc/passwd`

Awk

Definition

- Awk is a scripting language used for processing and displaying text. Awk can work with a text file or from the standard output.

Usage/formula

- `awk + options + {awk command} + file + file to save (optional)`

Examples

- Print the first column of every line

- `awk '{print $1}' ~/Documents/Csv/cars.csv`

- Print the last field of /etc/passwd file

- `awk -F '{print #NF}' /etc/passwd`

- Print the first and the last field of the /etc/passwd

- `awk -F '{print $1, " = " , $NF}' /etc/passwd`

- Print the first and 3 field of the /etc/passwd

- `awk -F '{print NR,$1,$3}' /etc/passwd`

- Print the first and the 4th field with a different field separator

- `awk -F '{OFS="="}{print $1,$4}' /etc/passwd`

- Start printing a file from a giving line(exclude the first 2 lines)

- `awk 'NR > 3 {print}' /etc/passwd`

- Convert the first field to upper/lower case

- `awk -F: '{print toupper($1)}' /etc/passwd`

- Print the length of a line(record)

- `awk '{print length($0)}' /etc/passwd`

- Print specific field based on a command output. For example, the size and file name

- `ls -lhF documents/ | awk 'BEGIN {printf "%s\t%s\n",:"Size","Name"} {print $5, "\t",$9}'`

Sed

Definition

- SED is a string editor that perform operations on files and standard output. For instant it can search and replace,insert,and deleted. By using SED you can edit files without opening them

Usage/formula

- `sed + options + sed script + file`

Examples

- Replacing a string in givin file globally(replace false for true)
 - `sed 's/false/true/g' ~/Documents/sample_files/Json.json`
- Replacing only the fourth occurrence per line in a file
 - `sed 's/false/true/4' joke.json`
- Replacing from the giving number occurrence to the rest occurrence in a file. Start at the second time the word appears and continue to till end of the file
 - `sed 's/false/true/3g' joke.json`
- Replacing string on a specific line number
 - `sed '3 s/false/true/' joke.json`
- Replace string on a range of lines
 - `sed '1,3 s/false/true/' joke.json`

How to use the pipe (|) for redirecting the output of a command to another.

- The **pipe** allows you to redirect the standard **output** of a command to the standard **input** of another.

Usage:

- `command_1 | command_2 | command_3 |.... | command_N`

Examples

- Use grep to look for a string in a particular man page
 - `man ls | grep "human-readable"`
- Display only the options of any command from its man page
 - `man ls | grep "^[[:space:]]*[:punct:]"`
- Display only the 2nd line in a file
 - `head -2 file.lst | tail -1`
- Display only the ip address from the output of the ip command

- `ip addr | grep -Eo '[:digit:]{1,3}\.[:digit:]{1,3}\.[:digit:]{1,3}\.[:digit:]{1,3}'`

How to save the output of a command to a file (>).

- The `>` operator in Linux is used to **redirect (save)** the output of a command into a file instead of displaying it on the screen.

Usage/syntax

- `command > filename`
 - `command`: Generates output
 - `>` redirect output
 - `filename` file where the output is saved
 - If the file does not exist, it will be **created**
 - If the file already exists, it will be **overwritten**

Examples

- **Save file list**
 - `ls > files.txt`
 - What happens:
 - `ls` lists all files in the current directory
 - **Output** is saved into files.txt
 - **Result**: files.txt contains the list of files
- **Save command output**
 - `date > today.txt`
 - What happens:
 - `date` shows current date and time
 - **Output** is saved into today.txt
 - **Result**: file contains current date/time
- **Save filtered results**
 - `grep "error" log.txt > errors.txt`
 - What happens:
 - `grep "error" log.txt` finds lines with "error"

- **Output** is saved into errors.txt
- **Result**: only error lines are stored
- **Important Note**
 - Using **>** will erase existing content in the file.
 - If you **do not want to overwrite** the file, use **>>** instead.

How to append the output of a command to a file.

- The **append operator** (>>) is used to **add the output of a command to the end of a file** without deleting what is already inside

Usage/syntax

- **command >>filename**
 - **command**: generates output
 - **>>** appends output
 - **filename** file where output is added
 - If the file **does not exist**, it will be created
 - If the file **exists**, new content is added **at the end**

Examples

- **Append text to a file**
 - **echo "Hello World" >> **notes.txt**
 - What happens:
 - Adds **"Hello World"** to the end of **notes.txt**
 - Result: existing content stays, new line is added
- **Append command output**
 - **date >> log.txt**
 -
 - What happens:
 - Current date/time is added to **log.txt** each time you run it
 - Useful for keeping logs over time
- **Append filtered results**
 - **grep "error" system.log >> errors.txt**
 - What happens:
 - Finds lines with **"error"** in **system.log**
 - Adds them to **errors.txt** without deleting previous entries

- Result: growing list of errors
- **Combine with pipes**
 - `ls | grep ".sh" >> scripts.txt`
 - What happens:
 - Lists files
 - Filters `.sh` files
 - Appends results to `scripts.txt`
 - Result: keeps adding new script names over time